Mid-Project Milestone Report

Project: SIMD-Accelerated Matrix Multiplication for Neural-Network Workloads
Group: 7

1. Accomplished Mid-Project Milestone Objectives

The following table cross-checks the deliverables promised in our project proposal for the Week 10 Mid-Project Update against the current state of our repository:

Promised in the proposal	Status
Sequential implementation	Done
NEON assembly, cache blocking, register reuse	Done
Scalar handling of leftover rows/cols	Done
Correctness: SIMD = sequential (including non-multiples of 4)	Done
Preliminary times and speedup	Done

1.1 Implementation Details & Deliverable Verification

- **Sequential implementation:** We established a strict scalar baseline in `src/matmul_seq.c`. The implementation uses a cache-friendly i-k-j loop order. To guarantee a pure baseline for accurate speedup calculations, the build disables compiler auto-vectorization with the `-fno-vectorize` and `-fno-slp-vectorize` Clang flags.

```c
#include "matmul.h"

/*
C = A x B, all size x size, row-major.
i-k-j order (row, inner, col): A[row][inner] multiplies the whole row
"inner" of B and adds it to row "row" of C.
*/
void matmul_seq(float *A, float *B, float *C, int size)
{
    // Initialize C to zero
    for (int i = 0; i < size * size; i++)
        C[i] = 0;

    // Perform matrix multiplication
    for (int row = 0; row < size; row++) {
        for (int inner = 0; inner < size; inner++) {
            float a_value = A[row * size + inner];
            for (int col = 0; col < size; col++)
                C[row * size + col] += a_value * B[inner * size + col];
        }
    }
}
```

Figure 1. Sequential baseline in `src/matmul_seq.c`, using the i-k-j loop order.

```
# -fno-vectorize -fno-slp-vectorize stop the compiler from adding SIMD by itself,
# so the sequential version really is sequential.
# -Iinclude lets the .c files find the headers in include/.
CC = clang
CFLAGS = -O2 -Wall -Iinclude -fno-vectorize -fno-slp-vectorize
```

Figure 2. Makefile flags that stop the compiler from auto-vectorizing the code.

- **NEON assembly, cache blocking, register reuse:** The core parallelization is fully operational in `src/matmul_simd.c`.
    - **Cache blocking:** The outer loops partition the matrices into 64 x 64 tiles, sized to fit within the L1 data cache of the Apple Silicon M-series architecture.

```c
void matmul_simd_tiled(float *A, float *B, float *C, int size)
{
    int TILE = 64; // L1 cache block size
    int size4 = (size / 4) * 4; // largest multiple of 4

    // init C to zero
    for (int i = 0; i < size * size; i++) C[i] = 0;

    // cache tiling - outer loops traverse matrices in blocks
    for (int rb = 0; rb < size4; rb += TILE) {
        int r_end = (rb + TILE < size4) ? rb + TILE : size4;

        for (int cb = 0; cb < size4; cb += TILE) {
            int c_end = (cb + TILE < size4) ? cb + TILE : size4;

            for (int ib = 0; ib < size; ib += TILE) {
                int i_end = (ib + TILE < size) ? ib + TILE : size;
```

Figure 3. Cache-tiling loops in `src/matmul_simd.c` that split the matrices into 64 x 64 tiles.

- **Register reuse:** The innermost operation is a handwritten AArch64 inline assembly block. It loads a 4 x 4 block of the output matrix into vector registers (`v16`–`v19`) before the inner accumulation loop begins, eliminating redundant memory reads and writes during the computation.
- **NEON assembly:** The micro-kernel uses the native `ld1`, `ld1r` (broadcast) and `fmla` (fused multiply-add) AArch64 instructions to compute four 32-bit floats at once per instruction.

```
__asm__ volatile (
    // load the 4x4 block of C into accumulators v16-v19
    "ld1 {v16.4s}, [%[c0]] \n"
    "ld1 {v17.4s}, [%[c1]] \n"
    "ld1 {v18.4s}, [%[c2]] \n"
    "ld1 {v19.4s}, [%[c3]] \n"

    "1: \n" // inner loop start (register reuse)

    // load 4 elements of B into v0, then move pointer down one row
    "ld1 {v0.4s}, [%[b]] \n"
    "add %[b], %[b], %[b_stride] \n"

    // load 1 scalar of A, broadcast to 4 lanes, increment pointer by 4 bytes
    "ld1r {v1.4s}, [%[a0]], #4 \n"
    "ld1r {v2.4s}, [%[a1]], #4 \n"
    "ld1r {v3.4s}, [%[a2]], #4 \n"
    "ld1r {v4.4s}, [%[a3]], #4 \n"

    // multipl-add (accum into C registers)
    "fmla v16.4s, v0.4s, v1.4s \n"
    "fmla v17.4s, v0.4s, v2.4s \n"
    "fmla v18.4s, v0.4s, v3.4s \n"
    "fmla v19.4s, v0.4s, v4.4s \n"

    // decrement tile counter, branch if not zero
    "subs %[iters], %[iters], #1 \n"
    "b.ne 1b \n"

    // store accumulated 4x4 C block back to memory
    "st1 {v16.4s}, [%[c0]] \n"
    "st1 {v17.4s}, [%[c1]] \n"
    "st1 {v18.4s}, [%[c2]] \n"
    "st1 {v19.4s}, [%[c3]] \n"
```

Figure 4. Inline NEON assembly micro-kernel. The 4 x 4 block of C stays in `v16–v19` for the whole k loop.

- **Scalar handling of leftover rows/cols:** The implementation is safe for irregular matrix dimensions. The C wrapper calculates the largest multiple of 4 (`size4`) for the vector operations. Because the vector loops stop at `size4`, the assembly kernel never reads or writes past the end of a row. Any remaining elements at the boundaries are routed to a trailing scalar cleanup loop, which computes each of them exactly once.

```
// scalar cleanup for edges and sizes not a multiple of 4
for (int row = 0; row < size; row++) {
    for (int inner = 0; inner < size; inner++) {
        float a_value = A[row * size + inner];
        // skip the already computed 4x4 blocks
        int col_start = (row < size4) ? size4 : 0;
        for (int col = col_start; col < size; col++) {
            C[row * size + col] += a_value * B[inner * size + col];
        }
    }
}
```

Figure 5. Scalar cleanup loop for the rows and columns left over after the 4 x 4 blocks.

- **Correctness (SIMD = sequential):** We built a dedicated testing harness (`tests/test_matmul.c`). The SIMD output is compared against the sequential output using an absolute floating-point tolerance of 1e-3. The automated tests sweep through irregular sizes that are not multiples of 4 or of the tile size (e.g., 3, 5, 7, 65, 67, 129) and verify that the results match the scalar baseline. The benchmark (`src/main.c`) also re-verifies correctness at every benchmarked size, including 255, before timing.

```
int verify_correctness(float *C_seq, float *C_simd, int size)
{
    for (int i = 0; i < size * size; i++) {
        if (fabs(C_seq[i] - C_simd[i]) > 1e-3) {
            printf("Mismatch at index %d: Seq=%f, SIMD=%f\n", i, C_seq[i], C_simd[i]);
            return 0;
        }
    }
    return 1;
}
```

Figure 6. `verify_correctness` in `src/utils.c`, which compares every element of the two outputs.

```
/*
Sizes 4 and up have full 4x4 blocks, so the assembly runs.
Odd sizes (5, 7, 67, ...) also run the scalar leftover code.
65 and 129 cross the 64 tile size.
*/
int test_sizes()
{
    int sizes[] = {1, 3, 4, 5, 7, 8, 11, 16, 63, 64, 65, 67, 128, 129};
    int num_sizes = sizeof(sizes) / sizeof(sizes[0]);
```

Figure 7. Matrix sizes swept by the correctness tests in `tests/test_matmul.c`.

- **Preliminary times and speedup:** The benchmark program (`src/main.c`) times each version using the C standard library `clock()` function, which measures CPU time. After a warm-up run, it runs matrix sizes from 64 x 64 up to 1024 x 1024, averages the execution time over 5 runs, and reports the throughput (GFLOPS) and the speedup.

```c
// benchmark SIMD
double total_time_simd = 0;
for (int run = 0; run < runs; run++) {
    clock_t start = clock();
    matmul_simd_tiled(A, B, C_simd, size);
    total_time_simd += (double)(clock() - start) / CLOCKS_PER_SEC;
}
double avg_simd = total_time_simd / runs;

// output
double gflops_base = 2.0 * size * size * size / 1e9;
printf("----------------------------------------\n");
printf("Matrix Size : %d x %d\n", size, size);
printf("Sequential  : %.6f s (%.2f GFLOPS)\n", avg_seq, gflops_base / avg_seq);
printf("SIMD + Tile : %.6f s (%.2f GFLOPS)\n", avg_simd, gflops_base / avg_simd);
printf("Speedup     : %.2fx\n", avg_seq / avg_simd);
```

Figure 8. Timing loop and result output in `src/main.c`.

1.2 Test Environment

All measurements for these preliminary tests were taken on a single machine, so the sequential and SIMD versions can be compared directly.

Item	Value
Machine	MacBook Air (M1, 2020)
Processor	Apple M1, 8-core CPU (4 performance + 4 efficiency cores)
Memory	8 GB unified memory
Operating system	macOS 15.2 (build 24C101)
Compiler	Apple clang version 16.0.0 (clang-1600.0.26.6)
Compiler flags	`-O2 -Wall -fno-vectorize -fno-slp-vectorize`
Performance core	Up to 3.2 GHz, 128 KB L1 data cache, 12 MB L2 shared by the performance cores
Threads	1 (single core, no multithreading)

1.3 Preliminary Results

Correctness: Running `make test` produced the output below. The hand-checked 2 x 2 case gives the exact expected result for both versions. For all 14 tested sizes (1, 3, 4, 5, 7, 8, 11, 16, 63, 64, 65, 67, 128, 129), every element of the SIMD output is within 1e-3 of the sequential output. These sizes include ones smaller than the vector width, ones that are not multiples of 4, and ones that cross the 64-element tile boundary.

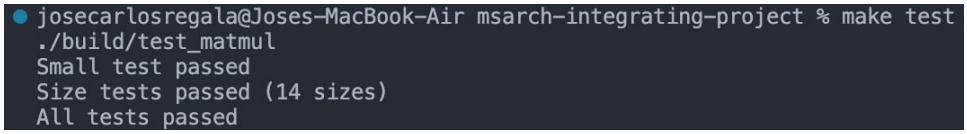


Figure 9. Output of `make test`: all correctness tests pass.

Execution time and speedup: Running `make run` produced the results below. Each time is the average of 5 runs after one warm-up run. Only the multiplication is timed (including zeroing C); matrix generation and checking are not. GFLOPS = $2N^3$ / (time × 10^9).

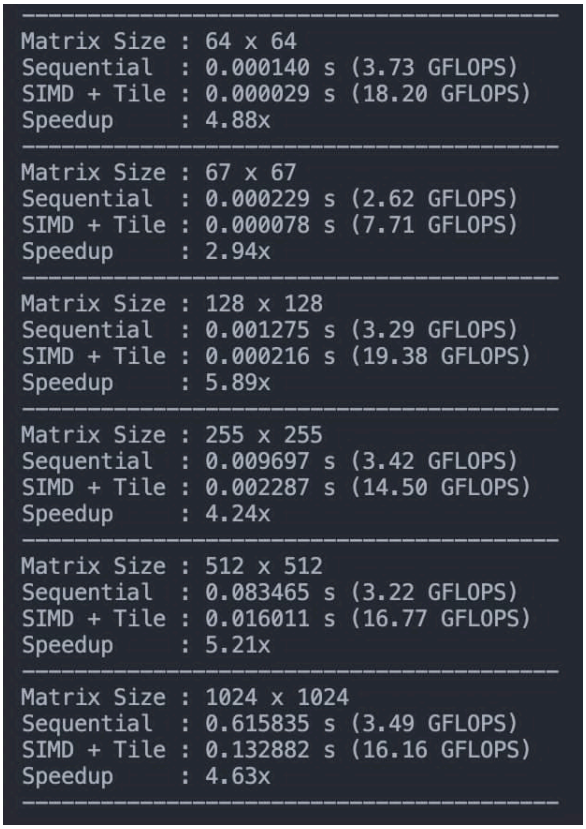


Figure 10. Output of `make run`: time, GFLOPS and speedup for each matrix size.

Matrix size	Sequential (s)	SIMD + Tile (s)	Seq. GFLOPS	SIMD GFLOPS	Speedup
64 x 64	0.000140	0.000029	3.73	18.20	4.88x
67 x 67	0.000229	0.000078	2.62	7.71	2.94x
128 x 128	0.001275	0.000216	3.29	19.38	5.89x
255 x 255	0.009697	0.002287	3.42	14.50	4.24x
512 x 512	0.083465	0.016011	3.22	16.77	5.21x
1024 x 1024	0.615835	0.132882	3.49	16.16	4.63x

Overall: The SIMD version was faster than the sequential version at every tested size, with speedups from 2.94x (67 x 67) to 5.89x (128 x 128). At five of the six sizes (64, 128, 255, 512 and 1024), the speedup is above the 4x we expected from vectorization alone. The sequential version runs at 2.62–3.73 GFLOPS, while the SIMD version reaches up to 19.38 GFLOPS.

Why the speedup can exceed 4x: The 4x in our proposal counts only the vector width. The SIMD kernel also keeps a 4 x 4 block of C in registers, so it does not load and store C on every multiply-add, and its four independent `fmla` instructions can run in parallel on the M1. Together, these explain the speedups above 4x.

Scaling with size: Apart from 67 x 67, the sequential speed stays at 3.22–3.73 GFLOPS all the way to 1024 x 1024 (12 MB of data, the size of the M1's L2 cache), so it is limited by scalar computation, not by memory. The SIMD version peaks at 19.38 GFLOPS (128 x 128) and drops only about 17% at 1024 x 1024, which suggests that the 64 x 64 tiles keep most of the data in cache.

Sizes that are not multiples of 4: 67 x 67 is the slowest case (7.71 GFLOPS, 2.94x). Its k loop splits into tiles of 64 and 3, so half of the kernel calls do only 3 iterations. About 9% of the work is also left to the scalar cleanup loop, and rows of 67 floats are not 16-byte aligned. The sequential version also slows down at this size, so part of the drop is not specific to SIMD. 255 x 255 shows a smaller dip (14.50 GFLOPS, 4.24x).

Room for improvement: Our best rate is about 19% of the M1's theoretical single-core NEON peak (about 102 GFLOPS). With only four accumulator registers and an `fmla` latency of about 4 cycles, the kernel can start about one `fmla` per cycle, which caps it near 25.6 GFLOPS. A larger micro-kernel with more accumulators would raise this limit.

Timing precision: These results are preliminary. They were timed with `clock()` on one machine, and the smallest sizes (under 0.25 ms per run) are the least precise.

1.4 Changes from the Proposal

1. **Tolerance:** The code checks an absolute difference of 1e-3 instead of the relative tolerance in the proposal. We will switch to a relative tolerance for the final submission.
2. **Square matrices only:** The functions multiply size x size matrices. This matches the proposal's test plan (M = K = N).
3. **Inline assembly:** The NEON kernel is handwritten assembly placed inside the C file with `__asm__`, not in a separate `.s` file.

2. Tasks to be Completed (Week 13 Final Submission)

To fulfill the final requirements for Milestone 3 (Final Submission and Demonstration), the following engineering, benchmarking and documentation tasks remain.

Engineering & Advanced Benchmarking

- **Apple Accelerate integration:** Implement a third execution baseline using Apple's Accelerate framework (`cblas_sgemm`) as a reference for the best speed available on this

machine. Accelerate may use Apple's AMX matrix coprocessor, not just NEON, so it is a practical upper reference, not a direct NEON-to-NEON comparison.

- **Cache tiling optimization:** Run empirical sweeps to tune the `TILE` size parameter (e.g., 32, 64, 128 and 256) and find the best tile size for the L1/L2 cache hierarchy.
- **Massive matrix stress testing:** Extend the list of matrix sizes in the benchmark program to include massive dimensions (e.g., 2048 x 2048 and 4096 x 4096) to evaluate the kernel's performance and cache eviction penalties under severe memory bandwidth limits.
- **Relative tolerance check:** Change `verify_correctness` to use a relative tolerance, as specified in the proposal.
- **Higher-resolution timer:** Replace `clock()` with a monotonic wall-clock timer (`clock_gettime(CLOCK_MONOTONIC)`) to improve precision at small sizes, where one run takes only tens of microseconds.
- **Investigating size-specific slowdowns:** Compare 67 with 68 and 255 with 256 to measure the cost of the short final k tile, the scalar cleanup and unaligned rows. Test sizes near 1024 that are not powers of two (e.g., 1000 and 1100) to check for cache conflicts from power-of-two row strides.

Analysis & Documentation

- **Algorithm documentation:** Write a formal discussion of the sequential and SIMD algorithms, covering the microarchitectural details of our cache blocking and register reuse.
- **Performance analysis:** Compile the final execution times, speedups and GFLOPS into charts (e.g., matrix size vs. GFLOPS) to show how performance changes as the matrices grow beyond the size of the CPU caches.
- **Limitation review:** Discuss the performance limitations found during testing and propose future improvements.
- **Explaining speedups above 4x:** Measure how much of the speedup comes from vectorization and how much from register reuse and tiling, for example by comparing against a version that vectorizes but does not keep C in registers.
- **Possible improvements:** Discuss a larger micro-kernel (for example 8 x 8, which uses more of the 32 NEON registers), merging a very short final k tile into the previous one, and data packing as used by BLIS-style libraries.
- **Final repository polish:** Finalize the GitHub README so that all source code, Makefile build instructions and final benchmark results are clearly presented.